

Convolutional Neural Networks

John Snyder

STAT 9340: Spring 2023

Convolutional Neural Network Motivations

So far we have seen how the DNN structure's flexibility is very useful for learning highly complex relationships.

- As data pass through the layers the nature of the relationship between input and output is learned.

Some data types will do much better if we enforce some structure in the DNN that we know exists.

- If we need to classify images, we should use a structure that can capture local patterns in neighborhoods of those images
 - ▶ Find edges, shapes, etc

Convolutional Neural Networks (CNNs) are inspired by the structure and function of the human visual system, where neurons in the visual cortex respond to specific features of an image.

Section 1

Convolutions

Definition

Basic Mathematical Definition:

$$(f * g)(t) = \int_{\mathbb{R}^d} f(\tau)g(t - \tau)d\tau = \int_{\mathbb{R}^d} f(t - \tau)g(\tau)d\tau.$$

Discrete convolution (e.g., $d = 2\dim$) [in practice, these are finite sums]

$$f[x, y] * g[x, y] = \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} f[i, j]g[x - i, y - j]$$

So, we can think of $f[\]$ as a “kernel” weight function that is applied to elements of the spatial image $g[\]$. We are familiar with this concept in terms of kernel smoothing; note that in kernel smoothing, the kernels must integrate to 1 (or sum, in the case of discrete data) - this is not the case for convolution - but, we sometimes still call them kernels. More often these are called “filters.”

1D Convolution Example

```
import numpy as np

# Define a 1D input vector
x = np.array([1, 7, 4, 2, 7, 3, 2])

# Define a 1D kernel
k = np.array([1, 0, -2])

# Define an output vector
output = np.zeros(len(x) - len(k) + 1)

# Apply the kernel to the input vector
for i in range(len(output)):
    output[i] = np.sum(k * x[i:i+len(k)])

# Print the output
print(output)

## [-7.   3. -10.  -4.   3.]
```

The second value in the result is obtained by

$$x_1 k_0 + x_2 k_1 + x_3 k_2 = 7 \cdot 1 + 4 \cdot 0 + 2 \cdot (-2) = 3$$

1d Convolution Padding

Note that the output of the previous example is smaller than the input. To get around this we can pad the ends of the actual input with 0.

```
import numpy as np

# Define a 1D input vector
x = np.array([1, 7, 4, 2, 7, 3, 2])

# Insert 0 at the beginning/end of the array
x = np.insert(x, [0, len(x)], 0)

# Define a 1D kernel
k = np.array([1, 0, -2])

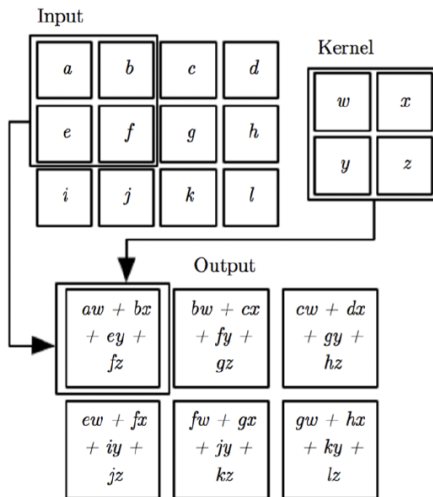
# Define an output vector
output = np.zeros(len(x) - len(k) + 1)

# Apply the kernel to the input vector
for i in range(len(output)):
    output[i] = np.sum(k * x[i:i+len(k)])

# Print the output
print(output)
```

```
## [-14.  -7.   3. -10.  -4.   3.   3.]
```

2d convolution



For the 2d convolution, we sweep through the input matrix similarly.

2d convolution examples - function

```
def convolve(img: np.array, kernel: np.array) -> np.array:
    # Assuming a rectangular image
    tgt_size = calculate_target_size(
        img_size=img.shape[0],
        kernel_size=kernel.shape[0]
    )
    k = kernel.shape[0]

    # 2D array of zeros
    convImage = np.zeros(shape=(tgt_size, tgt_size))

    # Iterate over the rows and columns
    for i in range(tgt_size):
        for j in range(tgt_size):
            subImage = img[i:i+k, j:j+k]
            convImage[i, j] = np.sum(np.multiply(subImage, kernel))

    return convImage
```

Meet our image

We will apply various kernels to the following image to see the features we can highlight



Sobel Kernel - Horizontal

This kernel highlights edges which run along the y axis.

```
kernel = np.array([[ -1,  -2,  -1],
                   [  0,   0,   0],
                   [  1,   2,   1]])

convolvedImg = convolve(img_square, kernel)
convolvedImg = np.clip(convolvedImg, 0, 255)
convolvedImg = convolvedImg.astype('uint8')
```



Sobel Kernal - Vertical

This kernel highlights edges on the x axis

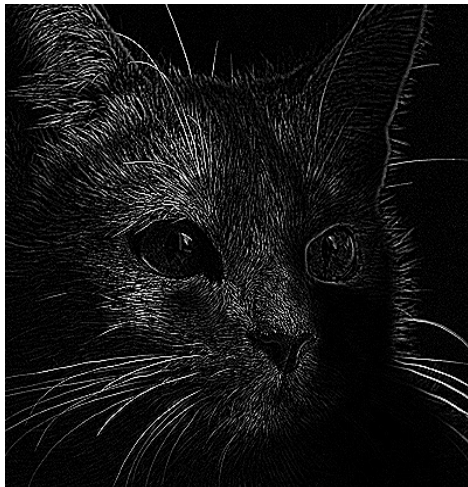
```
kernel = np.array([[ -1,  0,  1],
                   [ -2,  0,  2],
                   [ -1,  0,  1]])
convolvedImg = convolve(img_square, kernel)
convolvedImg = np.clip(convolvedImg,0,255)
convolvedImg = convolvedImg.astype('uint8')
```



Laplacian Kernel

This kernel does overall edge detection

```
kernel = np.array([[0, -2, 0],  
                  [-2, 8, -2],  
                  [0, -2, 0]])  
convolvedImg = convolve(img_square, kernel)  
convolvedImg = np.clip(convolvedImg, 0, 255)  
convolvedImg = convolvedImg.astype('uint8')
```



Sharpening Kernel

This kernel will sharpen images, why?

```
kernel = np.array([[0, 0, 0],  
                  [0, 2, 0],  
                  [0, 0, 0]]) - \  
          (1/9)*np.ones((3,3))  
convolvedImg = convolve(img_square, kernel)  
convolvedImg = np.clip(convolvedImg,0,255)  
convolvedImg = convolvedImg.astype('uint8')
```



Gaussian Blur

The following kernel is an approximation of a two dimensional normal distribution in its decay from the center

```
kernel = np.array([[1, 4, 6, 4, 1],  
                  [4, 16, 24, 16, 4],  
                  [6, 24, 36, 24, 6],  
                  [4, 16, 24, 16, 4],  
                  [1, 4, 6, 4, 1]])/256  
convolvedImg = convolve(img_square, kernel)  
convolvedImg = np.clip(convolvedImg,0,255)  
convolvedImg = convolvedImg.astype('uint8')
```



Two Random Kernels

Here we have two random kernels

```
np.random.seed(7)
randomkernel1 = np.random.uniform(-2,2, size= (3,3))
randomkernel2 = np.random.uniform(-2,2, size= (3,3))

print(randomkernel1)

## [[-1.69476684  1.11967517 -0.24636307]
## [ 0.89386071  1.91195805  0.15398348]
## [ 0.00448185 -1.71179547 -0.92624408]]
print(randomkernel2)

## [[-4.69996698e-04  7.16919984e-01  1.21495614e+00]
## [-4.76235467e-01 -1.73625461e+00 -8.47417603e-01]
## [ 1.63837411e+00 -1.14645859e+00 -1.91504153e-01]]
convolvedImg1 = convolve(img_square, randomkernel1)
convolvedImg1 = np.clip(convolvedImg1,0,255)
convolvedImg1 = convolvedImg1.astype('uint8')

convolvedImg2 = convolve(img_square, randomkernel2)
convolvedImg2 = np.clip(convolvedImg2,0,255)
convolvedImg2 = convolvedImg2.astype('uint8')
```



Section 2

Convolutional Neural Networks

How color images are handled

We essentially consider color images to be arrays, so that there is a multivariate value at each spatial location (pixel); e.g., in color images in RGB format, there are three values (associated with red, green and blue) that define the pixel color in the image with values ranging from 0-255.

- If you looked *really* closely at your computer monitor, you'd see this.

So, if the input is RGB, CNNs consider a multi dimensional convolution where the kernel aggregating is over location and the RGB channels. We can think of each layer of a CNN as having “height, width, and depth” (i.e., it is potentially 3-d).

Note, hidden layers in the CNN will always have depth > 1 (actually, this depth is usually quite large). Importantly, the filter operates on a relatively small part of the image (in x and y), but always operates across the entire depth; so, the weights (in 3-d) form a *tensor*.

Tensors in Tensorflow

The “Tensor” in Tensorflow is comes from the fact that the software is designed to handle computations on multi-dimensional arrays which can represent all kinds of inputs (images, audio, text, etc).

```
import tensorflow as tf
from tensorflow.keras.preprocessing.image import load_img, img_to_array

# Load the input image as RGB
img = load_img('squareImg.png', color_mode='rgb')

# Convert the image to a TensorFlow tensor
img_tensor = img_to_array(img)
img_tensor = tf.expand_dims(img_tensor, 0)

# Print the shape of the tensor
print(img_tensor.shape)

## (1, 400, 400, 3)
```

The first dimension corresponds to the batch size of the input. The next two describe the size of the image, and the last one the color channels

Convolution Calculation

The convolution operation involves sliding the kernel over the entire image. Even though embarrassingly parallel, this is still an expensive procedure. Convolution can be computed much more efficiently using the FFT.

Let $\mathcal{F}\{f\}, \mathcal{F}\{g\}$ be the Fourier transforms of f and g , respectively. Then,

$$\mathcal{F}\{f * g\} = \mathcal{F}\{f\} \cdot \mathcal{F}\{g\},$$

and

$$f * g = \mathcal{F}^{-1}\{\mathcal{F}\{f\} \cdot \mathcal{F}\{g\}\}.$$

Thus, one can do element-wise multiplication in Fourier space. This reduces the complexity of the problem from $\mathcal{O}(n^2)$ to $\mathcal{O}(n \log n)$ with n being the number of pixels.

How convolutions are used in DNNs

- We will initially set the kernel values as random and allow the DNN model to learn them.
- One of the strengths of the CNN approach is that it learns these kernel weights, but reduces the parameter space by assuming that there is only **one** set of weights for each convolution.
 - ▶ Although we will have potentially many convolutions for a given input.
- This ability to share weights is a huge dimension reduction in the parameter space.
- So, the CNN considers a convolution of the image with unknown weights that are learned.
- This convolution step is then followed by a “pooling layer” (or, “subsampling” or “down sampling”)

Pooling

The pooling layer considers a small rectangular block from the convolutional step and subsamples it in some way to produce a single output. There are several ways that one can do the pooling. E.g.,

- Block maximum (most popular by far)
- Block average
- L2-norm pooling: enforces a degree of spatial invariance
- Downsampling (ransomly subsampling one element per block)

Pooling is beneficial because

- Helps the network become robust to translations in input
- Reduces the dimensionality of the image

This pooling operation takes learned convolutions and reduces their dimensionality. This abstracted structure is then often put into another convolution/pooling operation in a CNN.

2 x 2 max pooling example



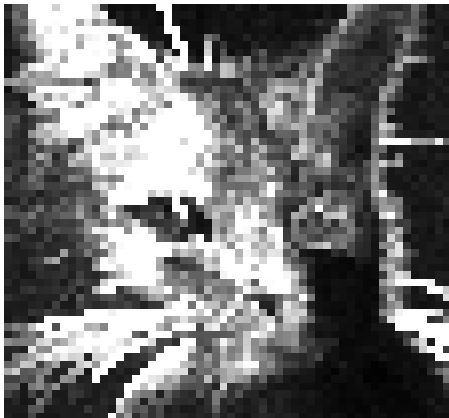
```
print(img.shape)
```

```
## (398, 398)
```

```
print(pooledOut.shape)
```

```
## (199, 199)
```

7 x 7 max pooling example



```
print(img.shape)
```

```
## (398, 398)
```

```
print(pooledOut.shape)
```

```
## (56, 56)
```

2 x 2 average pooling example



```
print(img.shape)
```

```
## (398, 398)
```

```
print(pooledOut.shape)
```

```
## (199, 199)
```

7 x 7 average pooling example



```
print(img.shape)
```

```
## (398, 398)
```

```
print(pooledOut.shape)
```

```
## (56, 56)
```

Notes on Pooling

When pooling, as we have seen, we will conduct this operation in a non overlapping fashion to reduce the dimensionality of the input.

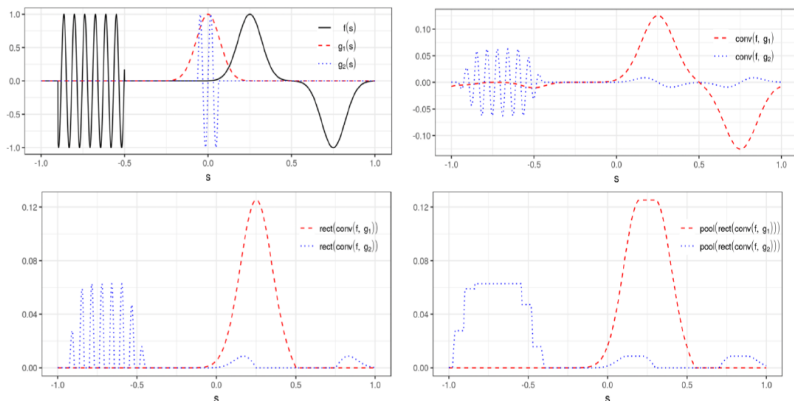
- This is known as the *stride* being the same dimension as the pooling.

There are (infrequent) cases where overlapping pooling regions can be useful

- Very high resolution imaging

Activation

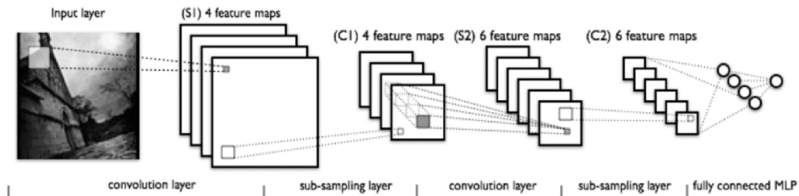
Note that the convolved images typically go through a ReLU rectification ($\text{ReLU}(x) = \max(0, x)$) before pooling. Consider a 1D example of convolution, rectifying, and pooling: black line - signal; red and blue lines are convolution filters



Overall CNN structure

The general structure of a CNN has alternating convolution layers followed by pooling (subsampling) layers; the last layers are fully connected.

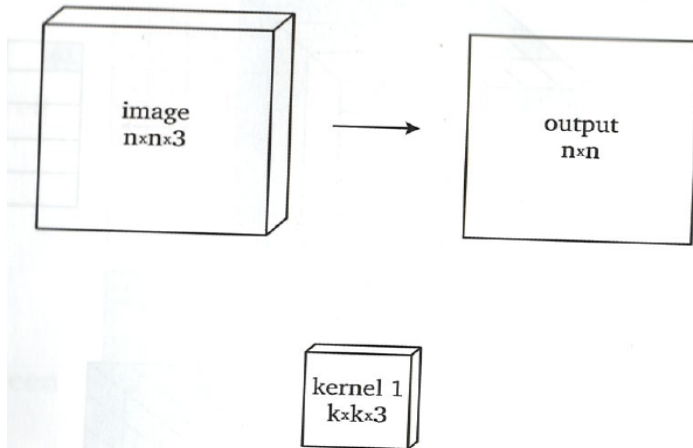
From A. Gibiansky's blog



- Multiple feature maps at the convolution stage created from multiple kernel weight matrices
- Typically, the convolved images go into a nonlinear activation function - usually, a rectified linear unit (ReLU) function:
- In general, pooling can occur across multiple feature maps; and, there are different poolings considered

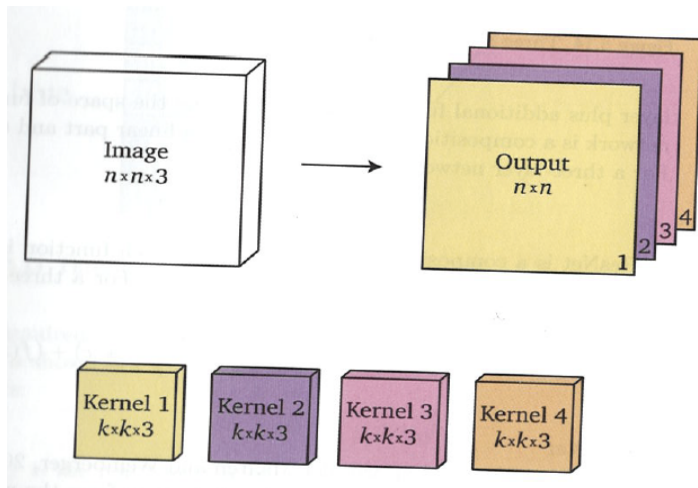
Convolutions happen over the “channels”

In another example of the feature map creation, we could consider this an “RGB” image where one kernel is applied and thus one feature map created.



Convolutions happen over the “channels”

Here we apply 4 kernels and get 4 feature maps



One more example

Consider the original LeCun CNN for the handwriting example:

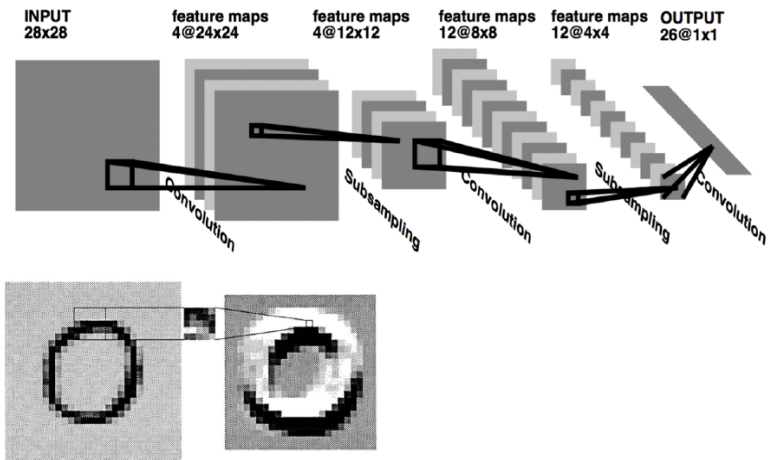


Figure 3: Input image (left), weight vector (center), and resulting feature map (right). The feature map is obtained by scanning the input image with a single neuron that has a local receptive field, as indicated. White represents -1, black represents +1.

CNN Formulation

The critical stage of the CNN that requires training is the convolution step (note, the pooling layers are typically specified and do no learning). Let's formally define the convolution step.

The p th filter (kernel) in the q th layer has parameters represented in the 3-dimensional tensor

$$W^{(p,q)} = \left[w_{ijk}^{(p,q)} \right],$$

where the indices i, j, k correspond to the positions along the height, width, and depth of the filter, respectively. The dimension of height, width and depth in the q th layer is given by L_q, B_q , and d_q , respectively. We say that the filter has dimension $F_q \times F_q \times d_q$. The feature maps in the q th layer are represented by the 3-dimensional tensor:

$$H^{(q)} = \left[h_{ijk}^{(q)} \right].$$

Note, $q = 1$ corresponds to the input layer.

CNN Formulation

The convolution operation from the q th layer to the $(q + 1)$ th layer is defined as:

$$h_{ijp}^{(q+1)} = \sum_{r=1}^{F_q} \sum_{s=1}^{F_q} \sum_{k=1}^{d_q} w_{rsk}^{(p,q)} y_{i+r-1,j+s-1,k}^{(q)}$$

for $i \in \{1, \dots, L_q - F_q + 1\}$, $j \in \{1, \dots, B_q - F_q + 1\}$, and $p \in \{1, \dots, d_{q+1}\}$. Note that the parameters for the convolutions for any given filter are the same for the entire feature map. This parameter sharing is important as it allows the presence of a shape in any part of the feature map to be processed the same way, regardless of its particular spatial location. The convolution step is then followed by an element-wise rectification (ReLU)

$$y_{ijp}^{(q+1)} = g \left(h_{ijp}^{(q+1)} \right). \quad [\text{typically, } g(\cdot) = \text{ReLU}(\cdot)]$$

CNN Formulation: Pooling

The pooling operation operates on $P_q \times P_q$ regions of each feature map. Note, unlike the convolution operator, the pooling operator applies to each feature map separately (i.e., we do not pool across the depth dimension). So, pooling produces d_q new feature maps, but the feature maps are typically much smaller. If a stride of 1 is used, each of the d_q pooled maps will be of dimension $(L_q - P_q + 1) \times (B_q - P_q + 1)$. But, it is more common to use a stride $S_q > 1$ when pooling, in which case the size of each map will be $((L_q - P_q) / S_q + 1) \times ((B_q - P_q) / S_q + 1)$. This helps reduce the size of the feature maps.

It is common to use max pooling with 2×2 filters with a stride of 2 (note, this implies there is no overlap in the different pooled regions - some people think it is helpful to have overlap as it helps prevent overfitting.)

Training the CNN

The biggest challenge with fitting a CNN model is the bookkeeping associated with all the weights.

The following is based on Gibransky (2016) where we consider a simple example with $d_q = 1$. Note that when we use the chain rule for these operations, we must sum the contributions of all expressions in which the variable occurs (this is due to the weight sharing).

$$\frac{\partial Q}{\partial w_{rs}^q} = \sum_i \sum_j \frac{\partial Q}{\partial h_{ij}^q} \frac{\partial h_{ij}^q}{\partial w_{rs}^q} = \sum_i \sum_j \frac{\partial Q}{\partial h_{ij}^q} y_{i+r-1, j+s-1}^{q-1}.$$

In this case, we must sum over all h_{ij}^q expressions in which w_{rs} occurs (this is due to weight sharing). Clearly, we need to know $\frac{\partial Q}{\partial h_{ij}^q}$, which we can get from the chain rule:

$$\frac{\partial Q}{\partial h_{ij}^q} = \frac{\partial E}{\partial y_{ij}^q} \frac{\partial y_{ij}^q}{\partial h_{ij}^q} = \frac{\partial E}{\partial y_{ij}^q} \frac{\partial}{\partial h_{ij}^q} \left(g \left(h_{ij}^q \right) \right).$$

Training the CNN

Thus, since we already know the error at the current layer $\frac{\partial E}{\partial y_{ij}^q}$, we can very easily compute the gradients $\frac{\partial Q}{\partial h_{ij}^q}$ at the current layer by just using the derivative of the activation function, $g'(h)$. Since we know the errors at the current layer, we now have everything we need to compute the gradient with respect to the weights used by this convolutional layer. Now, we must also propagate the errors back to the previous layer. Again, using the chain rule,

$$\frac{\partial E}{\partial y_{ij}^{q-1}} = \sum_r \sum_s \frac{\partial Q}{\partial h_{i-a+1,j-b+1}^q} \frac{\partial h_{i-a+1,j-b+1}^q}{\partial y_{ij}^{q-1}} = \sum_r \sum_s \frac{\partial Q}{\partial h_{i-a+1,j-b+1}^q} w_{rs}$$

This gives us the above value for the error at the previous layer. This looks slightly like a convolution. But, note it is applying to $h_{i-a+1,j-b+1}^q$ not $h_{i+a-1,j+b-1}^q$. So, this is a convolution where we have flipped the axes (transposed the feature map).

Training the CNN

We should also consider various strategies which have been shown empirically to improve performance.

- Padding
- Batch Normalization: After(?) activation, before pooling
- Data augmentation: Inject random transformations into the data

Additionally, because of data requirements necessary to train models like these, using *transfer learning* on publicly available models has become an accessible approach and software now accommodates this easily.

- 1 Load a pre trained model
- 2 Freeze weights of existing layers (optional)
- 3 Add an output which corresponds to your data
- 4 Train on your data

The amazing thing about this is that it works very well even if the original model was trained on data very different to yours.

Section 3

Implimentation and Specific Architectures

Key Tensorflow Layers for CNNs

There are a few layers in Tensorflow which facilitate fitting CNNs

- `tf.keras.layers.Conv2D`: Performs convolution, takes arguments
 - ▶ `filters`: Number of filters (or feature maps) in the layer.
 - ▶ `kernel_size`: Size of the kernel used. (e.g. (3,3))
 - ▶ `strides`: How far we move for each convolution (typically (1, 1))
 - ▶ `padding`: same for 0 padding valid for no padding.
 - ▶ `activation`: Activation function applied after convolution ('relu')
 - ▶ Others (initializers/regularizers)
- `tf.keras.layers.MaxPooling2D`: used for 2D max pooling on input.
 - ▶ `pool_size`: Size of the pooling window. (e.g. (3,3))
 - ▶ `strides`: Same as above. Should be the same as `pool_size` for non overlapping regions (typical). This is also the default.
 - ▶ `padding`: same for 0 padding valid for no padding.
- `tf.keras.layers.Flatten`: Used to convert the (final) 2d feature maps to a 1d feature vector
 - ▶ Used simply for reshaping before connecting to a dense layer used for the output.
 - ▶ Takes no arguments.

Quick Example

```
import tensorflow as tf

inputs = tf.keras.Input(shape=(28, 28, 1))
x = tf.keras.layers.Conv2D(32, (3, 3), padding='same', activation='relu')(x)
x = tf.keras.layers.BatchNormalization()(x)
x = tf.keras.layers.MaxPooling2D((2, 2))(x)
x = tf.keras.layers.Conv2D(64, (3, 3), padding='same', activation='relu')(x)
x = tf.keras.layers.BatchNormalization()(x)
x = tf.keras.layers.MaxPooling2D((2, 2))(x)
x = tf.keras.layers.Flatten()(x)
x = tf.keras.layers.Dense(64, activation='relu')(x)
outputs = tf.keras.layers.Dense(10, activation='softmax')(x)

model = tf.keras.Model(inputs=inputs, outputs=outputs)
```

ResNet

ResNet The deep residual neural network (ResNet) architecture (He et al., 2016a,b), introduced skip connections between consecutive layers to mitigate the vanishing gradient problem and allow for training deeper architectures (hundreds of layers).

In a neural network, the activations are the layer outputs:

$$z^{l+1} = f(W^l, z^l)$$

where f is the activation, W^l the weights of layer l , and z^l , the input activations to layer l . In a ResNet, the skip connections give the following form:

$$z^{l+1} = z^l + f(W^l, z^l)$$

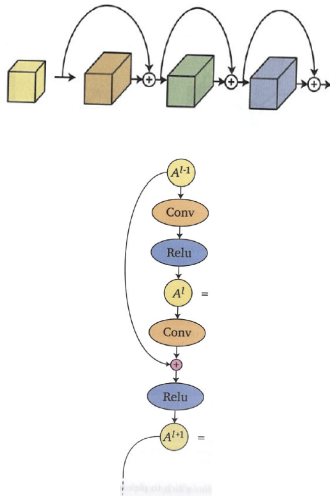
ResNet

This ultimately increases the space of functions we can fit with the same amount of parameters. For illustration, a 3 layer neural network could be seen as the composition

$$y = f(f(f(x))).$$

A ResNet is instead a composition of functions $f(x) + x$, and each layer represents a residual $f(x) - x$. For a three-layer ResNet we have

$$y = f(f(f(x)+x)+(f(x)+x))+f(f(x)+x)+(f(x)+x).$$



Code for an illustrative ResNet

```
def residual_block(x, filters):  
    # Save the input tensor  
    shortcut = x  
    # First convolutional layer  
    x = tf.keras.layers.Conv2D(filters, (3, 3), 1, padding='same')(x)  
    x = tf.keras.layers.Activation('relu')(x)  
    x = tf.keras.layers.BatchNormalization()(x)  
  
    # Second convolutional layer  
    x = tf.keras.layers.Conv2D(filters, (3, 3), padding='same')(x)  
  
    # Add the input tensor to the output of the residual block  
    x = tf.keras.layers.Add()([shortcut, x])  
    x = tf.keras.layers.BatchNormalization()(x)  
    x = tf.keras.layers.Activation('relu')(x)  
  
    return x
```

Code for an illustrative ResNet

```
input_shape = (400, 400, 3)

# Define the model
inputs = tf.keras.layers.Input(shape=input_shape)
x = tf.keras.layers.Conv2D(64, (3, 3), strides=(1, 1), padding='same')(inputs)
x = tf.keras.layers.Activation('relu')(x)
x = tf.keras.layers.BatchNormalization()(x)
x = tf.keras.layers.MaxPooling2D((3, 3), padding='same')(x)

x = residual_block(x, 64)
x = residual_block(x, 64)

x = tf.keras.layers.Flatten()(x)

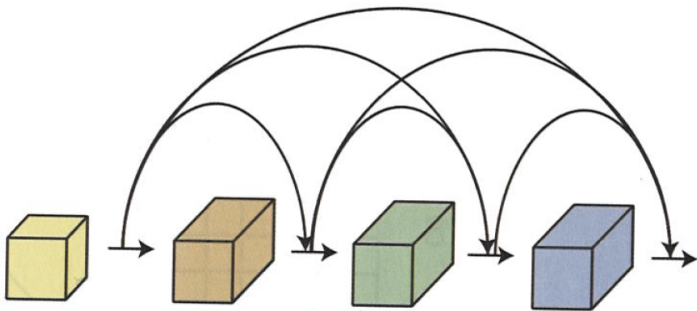
x = tf.keras.layers.Dense(units=128, activation='relu')(x)
outputs = tf.keras.layers.Dense(units=1, activation='softmax')(x)
```

One can also apply a CONV2D layer in the shortcut path to resize the input to residual_block to a different dimension

DenseNet

The DenseNet of Huang, Liu, van der Maaten and Weinberger, 2017 architecture concatenates the input x and output $f(x)$ of each layer to form the next layer $[f(x), x]$. Since a neural network is a composition of functions this results in a dense connection between each layer and its previous layers. For a three-layer network the composition is:

$$y = f([f([f(x), x]), [f(x), x]]), f([f(x), x]), [f(x), x])$$



Densenet code

```
def dense_block(inputs, num_layers, growth_rate):  
    """Defines a dense block with a specified number of layers"""  
    x = inputs  
    for _ in range(num_layers):  
        conv = tf.keras.layers.Conv2D(growth_rate, (3, 3), padding='same',  
                                       kernel_initializer='he_normal')(x)  
        norm = tf.keras.layers.BatchNormalization()(conv)  
        act = tf.keras.layers.ReLU()(norm)  
        x = tf.keras.layers.concatenate([x, act])  
    return x
```